

Problem Landscape and Success Criteria for One-User Offline-to-Online Personalized Policies

Research Position and Evaluation Contract

August 25, 2026

1 Setting

Evidence base. This synthesis uses both project PRISMA runs: 6,663 raw paper records before cross-run deduplication, plus 119 stored full-text review artifacts. We use the broad corpus to identify recurring problems, then cite representative primary papers for each problem. A citation below is evidence for that paper’s method or evaluation target, not evidence that our proposed policy already works.

One user is the unit of personalization. For user u , the offline history

$$H_u^{(0)} = \{\text{thoughts, conversations, corrections, preferences, task states, actions, tools, and outcomes}\}$$

produces an initial checkpoint $c_u^{(0)}$. After deployment, interaction batch $D_u^{(t)}$ is generated by the active checkpoint $c_u^{(t)}$. An update algorithm $\mathcal{A}$ emits

$$c_u^{(t+1)} = \mathcal{A}(c_u^{(t)}, H_u^{(0)}, D_u^{(1:t)}). \quad (1)$$

Every record names its user, time, task, generating checkpoint, action trace, feedback source, and training eligibility. A shared backbone or population prior is allowed, but another user’s raw history cannot silently enter user u ’s deployed branch.

A policy update is required. Retrieval, profiles, memory, and decoding control are strong baselines, not the learned object. OPU stores one user’s history in a private PEFT module [Tan et al., 2024]; P-RLHF learns user-conditioned policy parameters from preference pairs [Li et al., 2024]; and SPRInG updates one user adapter over chronological periods [Kim and Kim, 2026]. These are the closest parameter-learning precedents. CoSteer instead performs tuning-free local delta steering at decoding time [Lv et al., 2025], while FABLE updates a small external execution policy around a frozen host agent [Jin et al., 2026]. Both are important deployment baselines, but neither is the same claim as improving the user’s language-model policy weights.

The objective is constrained, not a single blended score. The method should improve future personalization while preserving task competence and safety:

$$\begin{aligned} \max_{\mathcal{A}} \quad & \mathbb{E}[U_{\text{pers}}(c_u^{(t+1)})] \\ \text{s.t.} \quad & \mathbb{E}[U_{\text{task}}(c_u^{(t+1)}) - U_{\text{task}}(c_u^{(t)})] \geq -\delta_{\text{task}}, \\ & \mathbb{E}[U_{\text{safe}}(c_u^{(t+1)}) - U_{\text{safe}}(c_u^{(t)})] \geq -\delta_{\text{safe}}, \\ & C_{\text{train}}, C_{\text{serve}}, C_{\text{feedback}} \text{ remain within declared budgets.} \end{aligned} \quad (2)$$

Task success and personalization are therefore co-primary measurements. One cannot compensate numerically for the failure of the other.

2 Exact-Match Filter

A paper belongs to our exact setting only when it satisfies all five conditions:

- A1.** it maintains a persistent policy for one identified user;
- A2.** it initializes that policy from the same user’s historical evidence;
- A3.** after deployment, new evidence from that user causes further updates;
- A4.** the policy acts in an agent task with actions, tools, or execution decisions; and
- A5.** the updated object includes parameters of the language-model policy, rather than only a profile, memory, reward model, decoder, or external controller.

Here *online* has a strict meaning: same-user evidence arrives after deployment and is followed by a parameter update. Fresh rollouts collected inside one centralized training job are useful online-training evidence, but they are not by themselves post-deployment continual personalization.

No cited method satisfies all five conditions. The problems below are therefore ranked by how directly they define the missing method, rather than by how often related keywords appear in prior work.

Table 1: Filtered priority of the twelve problems. Tier is about relevance to our target method; it does not imply that the cited papers share our setting.

Priority	Problems	Role in the project
Tier 1: method-defining	P4, P5, P6, P8, P9	Offline initialization, the offline–online handoff, continual retention, agent execution, and trajectory credit form the core algorithm.
Tier 2: necessary support	P1, P2, P12	User-evidence interpretation and future-behavior evaluation are required to make the core method meaningful and testable.
Tier 3: secondary claims	P7, P11	Feedback efficiency and serving efficiency matter, but should become headline claims only after dedicated comparisons pass.
Tier 4: optional scope	P3, P10	Cold start and simulator training are useful extensions, not requirements for the current known-user offline-to-online claim.

3 Problems Studied in This Setting

3.1 Learning the user

P1: identify preferences from sparse, mixed evidence. Aggregating feedback across users can erase minority or conflicting preferences [Li et al., 2024, Jang et al., 2023]. A single user also supplies heterogeneous evidence: explicit requests, implicit choices, edits, accepted outputs, and task outcomes do not share one label space. CoPL and PROPER borrow structure from related users to mitigate sparse histories [Choi et al., 2025, Zhang et al., 2025]; FSPO studies rapid adaptation from few preference examples [Singh et al., 2025]. The open question for us is whether one policy can use the target user’s complete history without reducing it to a textual profile.

Setting pointer. **Role:** Tier 2, necessary support. **One user:** mixed; the methods represent individual preferences, but several learn through shared users, groups, or population structure. **Offline initialization:** yes. **Post-deployment online update:** no. **Personalized agent:** no; the evidence concerns response preference or alignment. **LLM-policy parameter training:** mixed; P-RLHF, PROPER, and FSPO train generative-policy parameters, whereas CoPL primarily trains a personalized reward model. **Difference from us:** these papers infer preferences from fixed datasets; they do not update one deployed user’s agent policy from later task trajectories.

P2: separate persistent preference from transient context. A recent request may reflect a lasting preference, a temporary task constraint, or noise. SPRInG treats this as a drift-selection problem before updating a user adapter [Kim and Kim, 2026]. In an agent, the distinction is harder because failure

may come from task reasoning, tool execution, or a wrong preference assumption. A useful update must identify which part of the trace is evidence about the user.

Setting pointer. **Role:** Tier 2, necessary support. **One user:** yes; SPRInG maintains a distinct adapter and memory for each user. **Offline initialization:** yes, from period-0 user history. **Post-deployment online update:** no; later periods are a retrospective chronological simulation over a fixed corpus. **Personalized agent:** no; evaluation is personalized text generation. **LLM-policy parameter training:** yes, through per-user LoRA updates. **Difference from us:** SPRInG supplies a close continual-personalization component, but not live agent trajectories or a deployed offline-to-online agent loop.

P3: handle cold start without losing ownership. Sharing population structure improves few-shot adaptation, as explored by collaborative, group-level, and federated personalized models [Choi et al., 2025, Zhang et al., 2025, Zhu et al., 2026]. Yet unrestricted cross-user transfer conflicts with the one-user ownership boundary. The problem is to use a consented prior while keeping the target user’s updates, state, and evaluation distinct.

Setting pointer. **Role:** Tier 4, optional scope. **One user:** partial; a target user or client is represented, but the methods deliberately borrow structure across users. **Offline initialization:** yes. **Post-deployment online update:** no. **Personalized agent:** no. **LLM-policy parameter training:** mixed; PROPER and FedPDPO train personalized language-model components, while CoPL’s personalized object is a reward model. **Difference from us:** these are collaborative or federated offline personalization methods, whereas our current claim starts with a known user’s own history and does not require population-assisted cold start.

3.2 Moving from offline history to online learning

P4: build an offline initializer with online headroom. The offline checkpoint must already complete tasks, otherwise online interaction produces little useful feedback. It must also remain plastic. OPPU supplies a practical per-user initializer [Tan et al., 2024]; WebRL shows why a warm start matters for sparse-reward agent learning [Qi et al., 2025]. High offline fit alone is not enough if the resulting policy cannot improve from new evidence.

Setting pointer. **Role:** Tier 1, method-defining. **One user:** split; OPPU is per-user, while WebRL has no user identity. **Offline initialization:** yes in both lines. **Post-deployment online update:** no; WebRL’s fresh rollouts occur inside benchmark training, and OPPU is offline-only. **Personalized agent:** split; WebRL is an agent but not personalized, whereas OPPU is personalized but not an agent method. **LLM-policy parameter training:** yes. **Difference from us:** the two papers supply complementary halves, but neither initializes and then continually updates one user’s deployed agent policy.

P5: cross the offline–online boundary without a performance dip. Offline records were produced by older policies and may not cover actions preferred by the current policy. The LLM literature studies policy-generator synchronization [Lanchantin et al., 2025]; hybrid preference learning uses offline pairs to reduce online queries [Bose et al., 2024]; and policy expansion protects useful offline behavior during exploration [Zhang et al., 2023]. Our version must additionally preserve user identity, feedback provenance, and checkpoint lineage.

Setting pointer. **Role:** Tier 1, method-defining. **One user:** no. **Offline initialization:** yes. **Post-deployment online update:** no in our strict sense; “online” denotes fresh training-time rollouts or queries. **Personalized agent:** no. **LLM-policy parameter training:** mixed; Bridging updates an LLM policy, while hybrid preference optimization and policy expansion are broader optimization formulations. **Difference from us:** these methods solve the optimization handoff without same-user deployment, personal checkpoint lineage, or personalized agent evidence.

P6: adapt to drift without forgetting or chasing noise. Continual personalization must learn changed preferences while retaining stable ones. SPRInG selects high-drift interactions and keeps residual memory [Kim and Kim, 2026]. Asynchronous LLM training adds another source of staleness because behavior and learner policies diverge [Huang et al., 2026]. Success therefore requires both preference-level retention and valid use of old trajectories.

Setting pointer. **Role:** Tier 1, method-defining. **One user:** split; SPRInG is per-user, while Stable Asynchrony is not personalized. **Offline initialization:** yes for SPRInG; Stable Asynchrony has no personalized offline initializer. **Post-deployment online update:** no; both evaluations remain training-time or retrospective. **Personalized agent:** no. **LLM-policy parameter training:** yes. **Difference from us:** SPRInG addresses preference drift and Stable Asynchrony addresses stale LLM-RL trajectories, but neither combines both in a deployed personalized agent.

P7: learn quickly while minimizing user burden. Online feedback is expensive. The method should choose interactions that distinguish plausible user preferences, rather than requesting labels for easy or redundant examples. Hybrid preference optimization formalizes the offline-data versus online-query tradeoff [Bose et al., 2024]; FABLE is a pure-online method that learns a per-user execution policy as a contextual bandit [Jin et al., 2026]. In our setting, quick adaptation means fewer real interactions or corrections to reach a fixed future-task utility, not merely fewer optimizer steps.

Setting pointer. **Role:** Tier 3, secondary claim. **One user:** mixed; FABLE is per-user, while hybrid preference optimization is not. **Offline initialization:** split; hybrid optimization uses offline preferences, whereas FABLE has no learned offline initializer. **Post-deployment online update:** yes for FABLE, which is pure online; not in the target sense for the hybrid method. **Personalized agent:** yes for FABLE. **LLM-policy parameter training:** no for FABLE; it updates a small external execution policy around a frozen host agent. **Difference from us:** FABLE is the closest pure-online personalized agent baseline, but our target joins a historical LLM-policy initializer to later updates of that LLM policy.

3.3 Learning an agent policy

P8: preserve task completion while personalizing execution. Personalized agents must choose tools, memory, clarifications, action order, and response style while still finishing the task. PersonalWAB evaluates personalized web actions [Cai et al., 2025]; FingerTip 20K separates proactive suggestions from personalized mobile execution [Yang et al., 2026]; and pure-online FABLE separates an adaptive execution policy from frozen host-agent reasoning [Jin et al., 2026]. These works expose why response preference alone is insufficient: the same final answer may hide a bad or unwanted trajectory.

Setting pointer. **Role:** Tier 1, method-defining. **One user:** partial; all model personal behavior, but only FABLE maintains a persistent online execution policy for the individual user. **Offline initialization:** present only in the offline training/evaluation line, not in FABLE. **Post-deployment online update:** yes only for FABLE. **Personalized agent:** yes. **LLM-policy parameter training:** not in the exact combination; PersonalWAB and FingerTip do not establish persistent post-deployment per-user updates, and FABLE updates a non-LLM controller. **Difference from us:** these works establish the agent setting and its evaluation, but not one LLM policy that crosses from personal history into continued same-user parameter training.

P9: assign credit from long, partially observed trajectories. The user may provide one correction after many reasoning and tool steps. Sparse task outcomes say whether the task ended well, but not which step expressed the user’s preference. WebRL addresses sparse terminal feedback and policy drift for web agents [Qi et al., 2025]; our problem adds user-conditioned credit and delayed preference evidence.

Setting pointer. **Role:** Tier 1, method-defining. **One user:** no. **Offline initialization:** yes. **Post-deployment online update:** no; WebRL collects rollouts during a centralized training procedure.

Personalized agent: no; it trains an LLM web agent without persistent user identity. **LLM-policy parameter training:** yes. **Difference from us:** WebRL supplies sparse-reward agent-learning machinery, but its credit is task-conditioned rather than user-conditioned and does not come from a deployed personal stream.

3.4 Obtaining credible and affordable evidence

P10: use simulators without training to simulator artifacts. UserRL, UserVille, ProPerSim, and CollabLLM use simulated interaction to train or evaluate user-facing agents [Qian et al., 2025, Sun et al., 2025, Kim et al., 2025, Wu et al., 2025]. PersonalLLM also creates heterogeneous synthetic preference models [Zollo et al., 2024]. Simulation makes online learning affordable, but it can encode the simulator’s style and reward biases. Reward-model accuracy itself may fail to predict downstream personalization quality [Rezk et al., 2025]. A simulator can generate training experience; it cannot be the sole judge of a real-user claim.

Setting pointer. **Role:** Tier 4, optional scope. **One user:** no persistent target-user policy; simulators instantiate many users or personas for a shared system. **Offline initialization:** mixed and method-dependent. **Post-deployment online update:** no. **Personalized agent:** partial; several papers train or evaluate user-facing agents, but under simulated users. **LLM-policy parameter training:** mixed; these are primarily simulator, data, or training infrastructures rather than one common update algorithm. **Difference from us:** simulation can supply experience for our method, but cannot establish that one real user’s deployed LLM policy improves.

P11: make per-user deployment cheaper than repeated large-model use. Dedicated adapters create storage and serving overhead, while full online training adds accelerator cost. OPPU motivates lightweight user ownership [Tan et al., 2024]; CoSteer keeps context local while steering a cloud model [Lv et al., 2025]; and pure-online FABLE moves adaptation into a small external execution policy [Jin et al., 2026]. Our stronger efficiency question is whether a 4B, 7B, or 32B user policy can match or exceed a much larger generic model on user-conditioned utility at lower serving cost.

Setting pointer. **Role:** Tier 3, secondary claim. **One user:** yes; all three methods expose a user-specific component or context. **Offline initialization:** yes only for OPPU’s learned per-user adapter. **Post-deployment online update:** yes only for pure-online FABLE. **Personalized agent:** yes only for FABLE. **LLM-policy parameter training:** yes only for offline OPPU; CoSteer is tuning-free decoding-time local delta steering, and FABLE trains an external execution policy rather than the host LLM. **Difference from us:** each method satisfies a different efficiency slice, but none trains a compact personal LLM first offline and then from its deployed agent experience.

P12: evaluate future behavior rather than history recall. LongLaMP provides user-disjoint and chronological long-form tasks [Kumar et al., 2024]. PrefEval tests explicit and implicit preference following across long conversations [Zhao et al., 2025]. PERMA adds gradual preference formation, temporal interference, and task completion [Liu et al., 2026]; PRISM supplies individualized human feedback [Kirk et al., 2024]. These benchmarks show that retrieval accuracy is not the endpoint. The endpoint is correct behavior on an untouched future task.

Setting pointer. **Role:** Tier 2, necessary support. **One user:** yes at the data or evaluation level. **Offline initialization:** not applicable; these are datasets and benchmarks. **Post-deployment online update:** no. **Personalized agent:** mixed; PERMA has an agent-oriented environment, while LongLaMP, PrefEval, and PRISM primarily supply text or preference evidence. **LLM-policy parameter training:** no method is supplied. **Difference from us:** these resources can test our policy, but they do not define the offline-to-online update algorithm.

Table 2: The problem map. Each row is a separate empirical claim; success on one row does not imply success on another.

Problem	Main failure mode	Required evidence
Sparse preference inference	The model learns population averages or overfits a few labels.	Seen/unseen behavior, few-shot curves, and worst-user results.
Mixed personal history	Conversation text, task outcomes, and corrections are treated as interchangeable supervision.	Record-type ablations and held-out future behavior.
Offline initialization	The policy is competent but not personal, or personal but unable to complete tasks.	Separate offline task and personalization scores.
Offline–online handoff	Old evidence is stale and the first online updates reduce performance.	Transition curves, behavior-policy logging, and matched update budgets.
Drift and forgetting	The policy follows noise or erases earlier stable preferences.	Controlled shifts, backward transfer, recovery time, and false-update rate.
Feedback efficiency	Improvement requires too many labels, edits, or clarifications.	Utility versus real interactions and user time.
Agent credit assignment	A successful task receives the wrong personalized update.	Step or trace attribution ablations and trajectory-level evaluation.
Simulator validity	The policy optimizes simulator artifacts.	Held-out human agreement, calibration, and method-rank transfer.
Small-model efficiency	Personalization gains disappear under matched cost or stronger models.	Quality–cost Pareto curves and matched large-model baselines.
Ownership and privacy	Raw history leaks across users or to an undeclared service.	Data-flow audit, leakage tests, and explicit communication volume.

4 The Claim We Should Make

4.1 Central claim

A compact, user-owned language-model policy can be initialized from one person’s heterogeneous historical interactions and then improved from that person’s later agent-task evidence through real parameter updates, while preserving task success and safety.

This claim joins three pieces that prior work usually separates:

1. **Personal policy initialization.** The offline stage learns a persistent user-conditioned policy, rather than only retrieving a profile.
2. **Real offline-to-online adaptation.** Later evidence causes positive parameter updates and new loadable checkpoints.
3. **Agent validity.** Improvement is measured on both personalized behavior and task completion over untouched future tasks.

The contribution is therefore the *phase-coupling algorithm*: how historical evidence, fresh trajectories, feedback, replay, and checkpoint activation interact for one user. The contribution is not the existence of LoRA, retrieval, a user simulator, or an online optimizer by itself.

4.2 Secondary claims

These claims should appear only when their dedicated tests pass:

Fast adaptation. The method reaches a fixed utility with fewer real interactions than SPRInG-style selective SFT, naive cumulative fine-tuning, and online-only learning.

Trajectory advantage. Tool actions and verified outcomes improve future personalized execution beyond text-only user history.

Small versus large. A compact personalized policy beats a larger generic model on personalization while remaining task-non-inferior and cheaper to serve.

Simulation benefit. Simulator experience lowers real-user sample requirements without changing the ordering of methods under held-out human evaluation.

Privacy by design. The implementation satisfies a declared data-flow boundary and does not expose another user’s raw records. Local PEFT alone is not evidence for this claim.

5 Success Criteria

5.1 Hard validity gates

Before comparing scores, every reported update must pass:

1. the parent and candidate checkpoint hashes differ;
2. the candidate loads in a fresh worker and actually generates the evaluated trajectories;
3. offline, online-adaptation, validation, and future-test intervals are chronological and disjoint;
4. every training record has user, task, behavior-policy, checkpoint, timestamp, and feedback provenance;
5. failed runs, users, and rollbacks remain in the report;
6. the simulator, reward model, or LLM judge has not seen the future-test target or its answer.

Failure of any item invalidates the run rather than lowering its score.

5.2 Primary statistical gate

Let b be the strongest eligible baseline on the validation set. For each user-period cluster, evaluate the candidate and b on identical future tasks. Define

$$\Delta_{\text{pers}} = U_{\text{pers}}(c_{\text{candidate}}) - U_{\text{pers}}(c_b), \quad (3)$$

$$\Delta_{\text{task}} = U_{\text{task}}(c_{\text{candidate}}) - U_{\text{task}}(c_b). \quad (4)$$

The main claim passes only when

$$\text{LCB}_{95\%}(\Delta_{\text{pers}}) > 0 \quad \text{and} \quad \text{LCB}_{95\%}(\Delta_{\text{task}}) \geq -\delta_{\text{task}}. \quad (5)$$

The minimum meaningful personalization effect and δ_{task} must be fixed before final evaluation. Confidence intervals are clustered by user or task lineage, not by treating correlated turns as independent samples.

5.3 What “beat a large model” means

The large-model comparison must include:

1. a large generic model without personal history;
2. the same model with the full allowed history;
3. strong RAG or profile prompting;
4. a decoding-time method such as CoSteer when executable [Lv et al., 2025];
5. the compact offline-only and compact online-adapted policies.

Prompts, tools, history access, and task budgets must match. We may claim that the compact model beats the large model only if it improves personalization, meets task non-inferiority, and costs less. A win on preference style alone does not establish superior task capability.

Table 3: Claim-specific success criteria. The first four rows are required for the central claim; later rows authorize narrower secondary claims.

Claim	Measurement	Pass condition
Personalized improvement	Paired future-task Δ_{pers} against the strongest baseline.	Lower 95% bound is above zero and the point estimate exceeds the preregistered meaningful effect.
Task preservation	Paired task success or environment reward.	Lower 95% bound exceeds $-\delta_{\text{task}}$.
Value of online learning	Candidate versus frozen offline checkpoint and naive cumulative update.	Candidate passes equation (5) against both.
Real policy change	Parameter count, hashes, load receipt, and changed outputs.	Nonzero finite update; candidate is loaded for evaluation.
Fast adaptation	Area under the future-utility learning curve and interactions-to-target.	Higher clustered 95% interval for area, or fewer interactions to the same preregistered target.
Retention	Backward transfer on earlier stable preferences and tasks.	No task-margin violation; less forgetting than naive continual training.
Drift recovery	Utility after a controlled persistent preference change.	Faster recovery without elevated updates on transient-noise controls.
Safety	Severe violations and policy regressions under a fixed panel.	No increase in severe events; lower-severity change stays within its preregistered margin.
Small versus large	Compact personalized model versus large generic, RAG, and profile-enabled models under equal information.	Personalization superiority, task non-inferiority, and strictly lower serving cost all hold.
Simulator benefit	Human-simulator agreement, calibration, and method ranking.	Simulator-trained gain survives held-out human evaluation; simulation is never the only final judge.
Efficiency	Training GPU-hours, inference latency, memory, storage, tokens, API cost, and user-feedback count.	Candidate lies on the quality-cost Pareto frontier under matched budgets.
Ownership/privacy	Raw-record flow, communication bytes, cross-user leakage probes.	Declared boundary is met with no unauthorized raw-history transfer.

5.4 What quick adaptation means

Report utility after every fixed number of real interactions, not merely after epochs. For threshold τ ,

$$T_\tau = \min\{n : U_{\text{pers}}(c_u^{(n)}) \geq \tau \text{ and task/safety gates pass}\}. \quad (6)$$

Also report area under the learning curve, final utility, number of clarifications, user correction time, and rollbacks. A method is faster only if it reaches the same valid target with less user evidence or wall-clock cost.

5.5 What simulator success means

Simulation has three separate tests:

1. **Preference fidelity:** agreement and calibration on held-out real-user comparisons.
2. **Method fidelity:** rank correlation between methods under the simulator and under real-user evaluation.

Table 4: Minimum baseline families.

Family	Purpose	Representative implementation
No personalization	Measures the value of any user information.	Same-size base and large API model.
Prompt/profile/RAG	Tests whether parameter updates are necessary.	Full-history prompting, retrieval, and generated profile.
Offline per-user PEFT	Tests the value of the online phase.	OPPU [Tan et al., 2024].
Shared personalized policy	Tests one-user ownership against a population model.	P-RLHF, CoPL, or PROPER [Li et al., 2024, Choi et al., 2025, Zhang et al., 2025].
Continual per-user update	Tests the proposed transition and replay rule.	SPRInG and naive cumulative SFT [Kim and Kim, 2026].
Online-only	Measures the value of offline initialization.	Same optimizer initialized from the generic checkpoint.
Offline then naive online	Isolates the new coupling mechanism.	Sequential SFT/DPO/RL with no special replay or activation rule.
Weight-free online control	Tests whether policy training is worth its cost.	FABLE and CoSteer [Jin et al., 2026, Lv et al., 2025].
Large-model teacher	Tests compact-model competitiveness.	The CLI/API model that produced the offline trajectories.

3. **Transfer value:** fewer real interactions are needed after simulator pretraining than without it. Passing only simulator reward permits a simulator result, not a real-user personalization claim.

6 Experimental Comparison Set

Evaluation substrates. Use at least one chronological generation stream such as LongLaMP [Kumar et al., 2024] and one executable agent substrate such as PersonalWAB or FingerTip 20K [Cai et al., 2025, Yang et al., 2026]. PrefEval and PERMA provide additional long-horizon preference tests [Zhao et al., 2025, Liu et al., 2026]. User simulators can support training, but the final claim needs held-out human feedback or frozen real-user records. A result on one substrate is a feasibility result; a general claim requires at least two materially different substrates.

Replication. Although each deployed policy belongs to one user, a paper should repeat the same protocol over many independent users or user streams. Report per-user results, the median, the worst decile, and cluster-aware intervals. Use at least three seeds for feasibility and five for final claims.

7 Claims We Must Not Make

- Chronological offline replay is not post-deployment online learning.
- Better retrieval or profile recall is not evidence of a better policy.
- A simulator-only win is not evidence of real-user improvement.
- A changed loss value is not evidence that the candidate checkpoint was loaded.
- A personalized reward-model gain is not automatically a policy or task gain [Rezk et al., 2025].
- Local adapters do not by themselves prove privacy.
- Beating a large model on style does not mean beating it on task capability.
- Mean improvement cannot hide severe failures for a subset of users.

8 Recommended Paper Position

The strongest paper is not a survey of every personalization problem. It is an algorithm paper about the transition from *one user's historical evidence* to *one user's continually improving policy*. Data adaptation defines the offline input. Offline-to-online optimization is the method. Task-preserving future personalization is the primary result. Quick adaptation, simulator-assisted training, privacy, and small-model efficiency are valuable secondary claims, each earned by its own test.

References

- Avinandan Bose, Zhihan Xiong, Aadirupa Saha, Simon Shaolei Du, and Maryam Fazel. Hybrid preference optimization for alignment: Provably faster convergence rates by combining offline preferences with on-line exploration. *arXiv preprint arXiv:2412.10616*, 2024. URL <https://arxiv.org/abs/2412.10616>.
- Hongru Cai, Yongqi Li, Wenjie Wang, Fengbin Zhu, Xiaoyu Shen, Wenjie Li, and Tat-Seng Chua. Large language models empowered personalized web agents. In *Proceedings of the ACM Web Conference*, 2025. doi: 10.1145/3696410.3714842. URL <https://arxiv.org/abs/2410.17236>.
- Youngbin Choi, Seunghyuk Cho, Minjong Lee, MoonJeong Park, Yesong Ko, Jungseul Ok, and Dongwoo Kim. CoPL: Collaborative preference learning for personalizing LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. doi: 10.18653/v1/2025.emnlp-main.650. URL <https://aclanthology.org/2025.emnlp-main.650/>.
- Luke J. Huang, Zhuoyang Zhang, Qinghao Hu, Shangdong Yang, and Song Han. Stable asynchrony: Variance-controlled off-policy RL for LLMs. *arXiv preprint arXiv:2602.17616*, 2026. URL <https://arxiv.org/abs/2602.17616>.
- Joel Jang, Seungone Kim, Bill Yuchen Lin, Yizhong Wang, Jack Hessel, Luke Zettlemoyer, Hannaneh Hajishirzi, Yejin Choi, and Prithviraj Ammanabrolu. Personalized soups: Personalized large language model alignment via post-hoc parameter merging. *arXiv preprint arXiv:2310.11564*, 2023. URL <https://arxiv.org/abs/2310.11564>.
- Dian Jin, Zhi Zhang, Huichao Li, Yihe Pan, Rundong Huang, and Doudou Zhou. Personalizing large language model agents with small policy models. *arXiv preprint arXiv:2608.00215*, 2026. URL <https://arxiv.org/abs/2608.00215>.
- Jiho Kim, Junseong Choi, Woosog Chay, Daeun Kyung, Yeonsu Kwon, Yohan Jo, and Edward Choi. ProPerSim: Developing proactive and personalized AI assistants through user-assistant simulation. *arXiv preprint arXiv:2509.21730*, 2025. URL <https://arxiv.org/abs/2509.21730>.
- Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM personalization via selective parametric adaptation and retrieval-interpolated generation. *arXiv preprint arXiv:2601.09974*, 2026. URL <https://arxiv.org/abs/2601.09974>.
- Hannah Rose Kirk, Alexander Whitefield, Paul Roettger, Andrew Bean, Katerina Margatina, Juan Ciro, Rafael Mosquera, Max Bartolo, Adina Williams, He He, Bertie Vidgen, and Scott A. Hale. The PRISM alignment dataset: What participatory, representative and individualised human feedback reveals about the subjective and multicultural alignment of large language models. *arXiv preprint arXiv:2404.16019*, 2024. URL <https://arxiv.org/abs/2404.16019>.
- Ishita Kumar, Snigdha Viswanathan, Sushrita Yerra, Alireza Salemi, Ryan A. Rossi, Franck Dernoncourt, Hanieh Deilamsalehy, Xiang Chen, Ruiyi Zhang, Shubham Agarwal, Nedim Lipka, Chien Van Nguyen, Thien Huu Nguyen, and Hamed Zamani. LongLaMP: A benchmark for personalized long-form text generation. *arXiv preprint arXiv:2407.11016*, 2024. URL <https://arxiv.org/abs/2407.11016>.
- Jack Lanchantin, Angelica Chen, Janice Lan, Xian Li, Swarnadeep Saha, Tianlu Wang, Jing Xu, Ping Yu, Weizhe Yuan, Jason Weston, Sainbayar Sukhbaatar, and Ilia Kulikov. Bridging offline and online reinforcement learning for LLMs. *arXiv preprint arXiv:2506.21495*, 2025. URL <https://arxiv.org/abs/2506.21495>.
- Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Leqi Liu. Personalized language modeling from personalized human feedback. *arXiv preprint arXiv:2402.05133*, 2024. URL <https://arxiv.org/abs/2402.05133>.

- Shuochen Liu, Junyi Zhu, Longhui Shu, Jun Lin, Yuhao Chen, Haotian Zhang, Chao Zhang, Derong Xu, Jia Li, Bo Tang, Zhiyu Li, Feiyu Xiong, Enhong Chen, and Tong Xu. PERMA: Benchmarking personalized memory agents via event-driven preference and realistic task environments. *arXiv preprint arXiv:2603.23231*, 2026. URL <https://arxiv.org/abs/2603.23231>.
- Hang Lv, Sheng Liang, Hao Wang, Hongchao Gu, Yaxiong Wu, Wei Guo, Defu Lian, Yong Liu, and Enhong Chen. CoSteer: Collaborative decoding-time personalization via local delta steering. *arXiv preprint arXiv:2507.04756*, 2025. URL <https://arxiv.org/abs/2507.04756>.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2411.02337>.
- Cheng Qian, Zuxin Liu, Akshara Prabhakar, Jielin Qiu, Zhiwei Liu, Haolin Chen, Shirley Kokane, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. UserRL: Training interactive user-centric agent via reinforcement learning. *arXiv preprint arXiv:2509.19736*, 2025. URL <https://arxiv.org/abs/2509.19736>.
- Fady Rezk, Yuangang Pan, Chuan-Sheng Foo, Xun Xu, Nancy Chen, Henry Gouk, and Timothy Hospedales. The reward model selection crisis in personalized alignment. *arXiv preprint arXiv:2512.23067*, 2025. URL <https://arxiv.org/abs/2512.23067>.
- Anikait Singh, Sheryl Hsu, Kyle Hsu, Eric Mitchell, Stefano Ermon, Tatsunori Hashimoto, Archit Sharma, and Chelsea Finn. FSPO: Few-shot optimization of synthetic preferences personalizes to real users. *arXiv preprint arXiv:2502.19312*, 2025. URL <https://arxiv.org/abs/2502.19312>.
- Weiwei Sun, Xuhui Zhou, Weihua Du, Xingyao Wang, Sean Welleck, Graham Neubig, Maarten Sap, and Yiming Yang. Training proactive and personalized LLM agents. *arXiv preprint arXiv:2511.02208*, 2025. URL <https://arxiv.org/abs/2511.02208>.
- Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. doi: 10.18653/v1/2024.emnlp-main.372. URL <https://arxiv.org/abs/2402.04401>.
- Shirley Wu, Michel Galley, Baolin Peng, Hao Cheng, Gavin Li, Yao Dou, Weixin Cai, James Zou, Jure Leskovec, and Jianfeng Gao. CollabLLM: From passive responders to active collaborators. *arXiv preprint arXiv:2502.00640*, 2025. URL <https://arxiv.org/abs/2502.00640>.
- Qinglong Yang, Haoming Li, Haotian Zhao, Xiaokai Yan, Jingtao Ding, Fengli Xu, and Yong Li. FingerTip 20K: A benchmark for proactive and personalized mobile LLM agents. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2507.21071>.
- Haichao Zhang, Wei Xu, and Haonan Yu. Policy expansion for bridging offline-to-online reinforcement learning. *arXiv preprint arXiv:2302.00935*, 2023. URL <https://arxiv.org/abs/2302.00935>.
- Linhai Zhang, Jialong Wu, Deyu Zhou, and Yulan He. PROPER: A progressive learning framework for personalized large language models with group-level adaptation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. doi: 10.18653/v1/2025.acl-long.800. URL <https://arxiv.org/abs/2503.01303>.
- Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. Do LLMs recognize your preferences? evaluating personalized preference following in LLMs. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2502.09597>.

- Kewen Zhu, Liping Yi, Zhiming Zhao, Zhuang Qi, Han Yu, and Qinghua Hu. FedPDPO: Federated personalized direct preference optimization for large language model alignment. *arXiv preprint arXiv:2603.19741*, 2026. URL <https://arxiv.org/abs/2603.19741>.
- Thomas P. Zollo, Andrew Wei Tung Siah, Naimeng Ye, Ang Li, and Hongseok Namkoong. PersonalLLM: Tailoring LLMs to individual preferences. *arXiv preprint arXiv:2409.20296*, 2024. URL <https://arxiv.org/abs/2409.20296>.